

Lovetree: Product Vision & Engineering Blueprint

처음 사랑에 빠진 순간부터 팬이 되어가는 경로를 엮는
단일 출처 문서 (Single Source of Truth)



[CONFIDENTIAL] For Lovetree PMs, Designers, and Engineers

단순한 저장소가 아닌, 감정의 러브트리

Lovetree는 처음 사랑에 빠진 순간부터
팬이 되어가는 경로를 영상과 메모로 연결해
남기는 팬 감정 기록 제품입니다.



Not: 단순한 영상 리스트, 일반 커뮤니티 피드



But: 나만의 덕질 경로, 시각화된 흐름

기존 도구의 한계와 Lovetree의 해결책

기록의 대상
저장 형태
공간의 성격
회고(Revisit)

[기존 도구]



영상 링크 자체



파편화된 리스트나 폴더



모두가 섞이는
공용 커뮤니티



무엇을 저장했는지
찾기 어려움

[Lovetree]



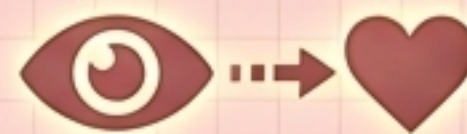
영상 내 특정 시점과
그때의 '감정'



시간순으로 이어지는
유기적인 나무(트리) 구조



완벽한 개인 기록 후,
원하는 대상과만 공유



감정이 커져온 경로가
한눈에 보임

4 Product Pillars (핵심 가치)



First Moment First

가장 중요한 것은 '첫 순간'. 누구를 처음 보고 떨렸는지, 어떤 장면에서 마음이 움직였는지 기록하는 것이 1순위.



Connected Love Path

파편화된 나열 금지. 첫 순간, 다음 영상, 리액션, 굿즈 소비까지 하나의 경로(Path)로 연결.



Emotion Over Archive

단순 링크 아카이빙은 불충분. 영상의 특정 시점(Timestamp)과 '왜 사랑에 빠졌는지'에 대한 감정 메모가 필수.



Private First, Share Second

개인의 솔직한 감정 기록이 먼저. 그 후 완성된 트리를 다른 사람을 '입덕'시키기 위해 공유.

Platform UX Direction: 이중 구조

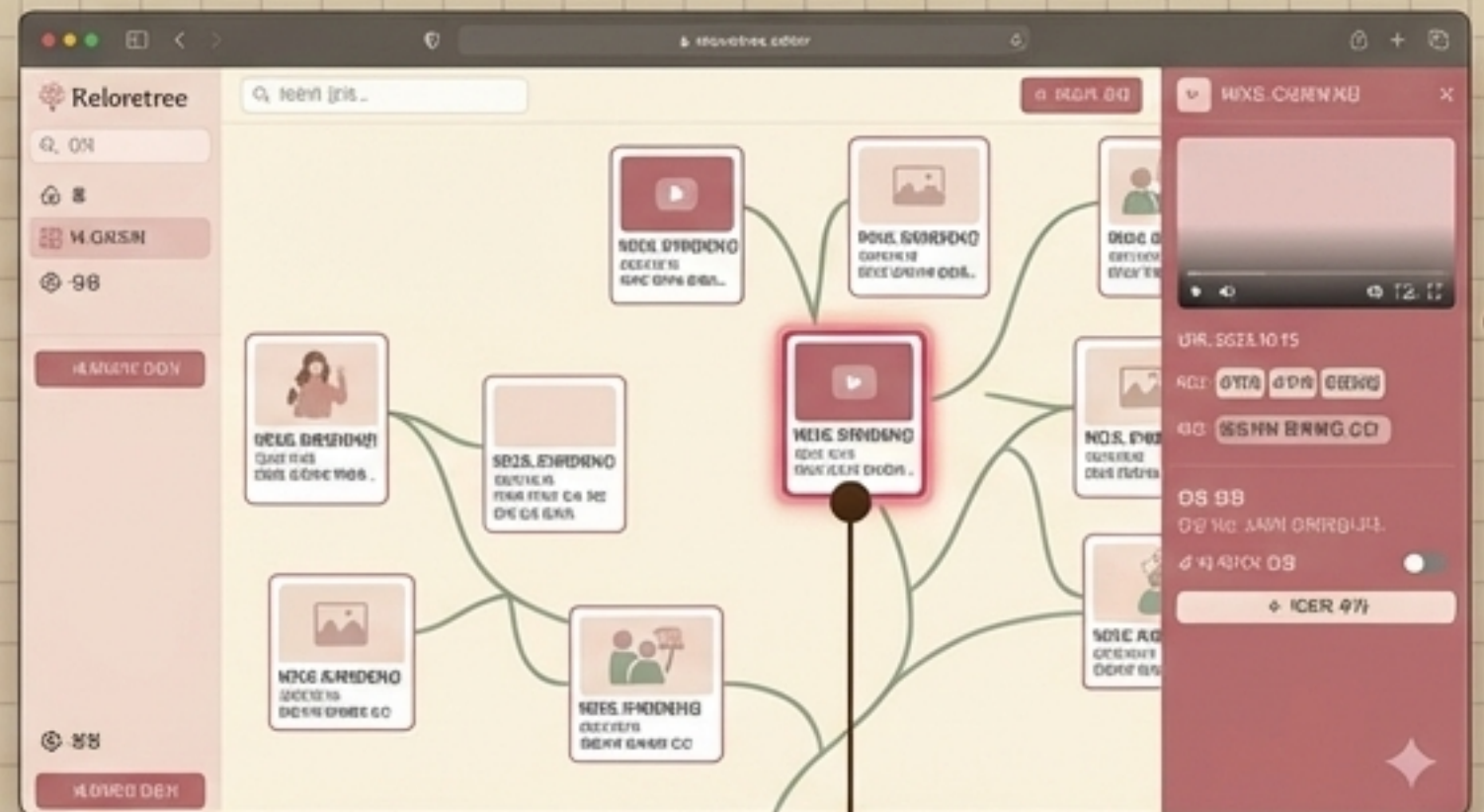
Mobile (빠른 포착과 쉬운 감상)

링크 붙여넣기
와 공유하기로
즉각적인 진입



세로 스크롤
중심의 단순화된
가지 구조 감상

Desktop (구조 연결과 정교한 편집)



드래그 앤 드롭 중심의
브랜치(가지) 연결

감정 기록 보드(Board)
형태의 조망과 편집 최적화

Flow A: 순간의 포착 (Step 1~3)



Step 1: Entry (진입)

모바일 공유하기 또는
링크 붙여넣기로 즉각 진입.



Step 2: Source Capture (영상 포착)

영상 섬네일 및 플랫폼 유효성 확인.



Step 3: Moment Capture (감정 기록)

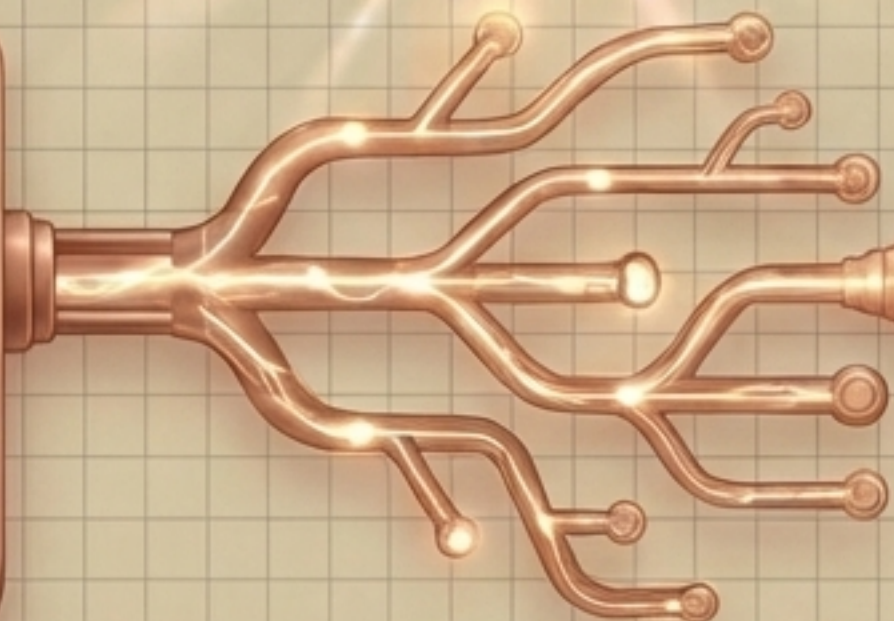
영상 전체가 아닌 '특정 시점(Timestamp)',
짧은 제목, 감정 태그, 그리고 감정 메모 기록.
메모가 없으면 가치가 없다.

Flow A: 경로의 연결과 전파 (Step 4~6)



Step 4: Connection (연결)

새 순간을 첫 루트로 삼거나,
기존 기록의 뒤에 연결하여
경로(Path) 형성.



Step 5: Revisit (회고)

구조화된 전체 트리를 조망하며
감정이 커져온 흐름을 다시 감상.



Step 6: Share (공유 및 전파)

완성된 트리의 링크를 타인에게
공유하여 공감/댓글 등의 반응을 얻고
새로운 팬을 '입덕'시킴.

MVP Scope Boundary (무엇을 만들고 무엇을 버릴 것인가)

In-Scope (Must Have)

- ✓ Firebase Auth 기반 인증
- ✓ 트리 생성 및 메모 노드 연결 (URL, 시점, 메모)
- ✓ 모바일/PC 트리 감상
- ✓ 링크 기반 공유 및 커뮤니티 최소 반응 (공감/댓글)

Out-of-Scope (Drop for now)

- ✗ 프리미엄 과금 모델
- ✗ AI 자동 태깅 및 복잡한 추천 알고리즘
- ✗ 굿즈/공연 결제 연동 및 다중 트리 템플릿

Database Architecture: The 'Firestore' Illusion

코드에 Firestore라는 이름이 남아 있지만,
실제 데이터 저장소는 Neon/PostgreSQL입니다.

Databased UI Surface

`firebase.firestore()`

표면층: Client Code
겉모습 (Illusion)

중간 변환층: Legacy Shim
& Netlify Functions
`firebase-firestore-compat.js`.
필터/파이프 역할.
하위 호환을 위한 라우팅.

중간 변환층: Legacy Shim
& Netlify Functions
`firebase-firestore-compat.js`.
필터/파이프 역할.
하위 호환을 위한 라우팅.

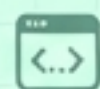
심연층: Neon PostgreSQL DB
실제 데이터가 저장되는
단단한 코어.

Data Naming & Access Rules (신규 개발자 필수 규칙)

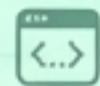
16:9 technical manual



[DO] 공식 진입점 사용



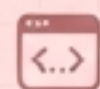
브라우저: 'src/postgres-client-browser.js' 로드 후
'window.postgresDB' 사용.



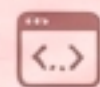
서버: 'netlify/functions/_lib/db-api.js'
내부 함수 사용.



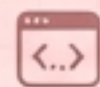
[DON'T] 절대 금지 사항



'firebase-firestore-compat.js' 직접
참조 금지 (내부 로드됨).



신규 코드 함수명/주석에 'Firestore' 추가
금지 (레거시 코드와 구분 불가).



'firebase.firestore.FieldValue' 직접
호출 금지 (기존 에디터 코드에만 허용).

The Editor Architecture Guardrails

Warning: Editor 영역('editor.html')은 일반 페이지 리팩터링 트랙에서 제외되어 격리 관리됩니다.

Layer 1: E2E / UI Presence

('smoke.spec.js') 표준 페이지의 CSS/HTML 일치 여부 확인.

Layer 2: Editor Shell Integrity

('editor-smoke.spec.js') 공통 로직(Shared) 수정 시 에디터 초기화 및 헬이 깨지지 않는지 검증.

Editor Core

Layer 3: Data Consistency Shim

('editor-fieldvalue.spec.js') 'FieldValue' 관련 코드 수정 시 소스 패턴과 네트워크 Payload가 정확히 일치하는지 무결성 검증.

Decision Checklist (의사결정 나침반)

Check 1: 이 기능이 '첫 순간 기록'을 더 쉽게 만드는가?

Check 2: 이 기능이 파편화된 리스트가 아닌 '연결된 사랑의 경로'를 보여주는가?

Check 3: 이 기능이 단순 아카이빙을 넘어 '감정(메모, 태그)'을 남기게 하는가?

Check 4: 이 기능이 내 기록을 '다시 보기 쉽게' 만드는가?

Check 5: 이 기능이 공유와 전파(입덕)에 '실제 도움'이 되는가?

Note: 이 다섯 질문에 답할 수 없다면, 그 기능은 기각됩니다.

The Handoff Rule & Source of Truth

Priority 1: PRODUCT_IDENTITY.md

제품 방향이 헛갈릴 때 기준점 (가장 높은 권위).

Priority 2: MVP_SCOPE.md

구현 범위가 헛갈릴 때의 기준점.

Priority 3: USER_FLOW.md

단일 화면이 아닌 '기록에서 공유까지의 흐름'을 잃었을 때.

Priority 4: DATA_NAMING_RULE.md

변수명, DB 접근 아키텍처가 헛갈릴 때.

Final Rule: 외부 작업자(AI 에이전트 포함)에게 Handoff 시,
'페이지 단위'가 아닌 반드시 '흐름(Flow) 단위'로 문서를 전달할 것.

